GMXTRANSPLANT

MD System Builder

Version 0.1.3

Guide to the desktop application and command-line workflows.

Generated 2026-10-13.

Contents

Overview 4
- Choose a task 4
- Install 4
- Desktop application 6
- Quick start 6
- Prepare your inputs 7
- Optional OpenMM minimization 9
- Assembly checks and outputs 11
- Limitations 11
- Runnable protein insertion example 11
- Development 12

Desktop application 13
- Open the application 13
- Try an included example 13
- Set up your own system 14
- Find and review your results 14
- Common questions 15
- Platform help 15

CHARMM-GUI transplant mode 16
- Detection and selection 16
- Optional YAML 17
- Alignment, topology and outputs 17
- GUI, examples and visualization 18

Add binder mode 19
- Inputs 19
- How placement works 19
- Output 21
- All options 22
- Desktop application 22
- Limitations 22

Membrane replacement 23
- Quick start 23
- Inputs 23
- What happens 23
- Fill the host box completely: membrane-patch 24
- Output 24
- Equilibration 25
- All settings 25
- Limits 26

System-edit modes 27
- Settings every mode shares 27
- salt 27
- protonate 27
- mutate 28
- hmr 28
- resize 28
- flood 29
- oligomer 29
- The equilibration folder 29

The GROMACS check 29
Validation 30
Comparison views 31
Portable and offline use 32

Overview

GMXtransplant builds membrane-protein systems from prepared structures and GROMACS topologies. Give it two CHARMM-GUI folders to transplant proteins and bound ligands automatically, or choose a workflow to insert a protein complex, replace a ligand, restore experimental cholesterol, place a binder near a membrane protein, or replace the membrane with an equilibrated POPC bilayer. Further modes change one prepared system: its salt, protonation states, residues, hydrogen masses, box height, ligand copies in the water, or the number of protein copies.

Use the desktop application to select inputs, run the included examples, and inspect results. The gmxtransplant command-line tool supports the same tasks for scripts and remote computers. Each desktop run saves its configuration as run.yaml in the output folder.

All molecular components need compatible parameters before assembly. Inspect the resulting coordinates and topology before simulation. For the two-folder workflow, see the mode guide (docs/guides/CHARMPROT.md), example (examples/charmprot/README.md), and PyMOL/VMD visualization guide (docs/guides/VISUALIZATION.md).

Choose a task

Task	Command option	Example configuration
Replace proteins and ligands using only two CHARMM-GUI folders	`--mode charmprot`	`charmprot.yaml`
Insert a protein or protein-ligand complex into a prepared environment	`--mode protein`	`protein_replace.yaml`
Replace a bound ligand while retaining the protein	`--mode lig`	`ligand_replace.yaml`
Restore experimental cholesterol and adjust leaflet composition	`--mode chl`	`cholesterol_restore.yaml`
Add a ligand or protein binder in the water above/below a membrane protein (guide (docs/guides/ADDBINDER.md))	`--mode addbinder`	`addbinder.yaml`
Replace the membrane with a well-equilibrated bilayer, keeping the OPM placement: pure POPC by default, or any bilayer GRO you give (guide (docs/guides/MEMBRANE.md))	`--mode membrane`	`membrane.yaml`
Prepare a restrained OpenMM minimization folder for an assembled system	`prepare-minimization`	`minimization.yaml`

More modes, each changing one prepared system (guide (docs/guides/EDIT_MODES.md)); in the desktop application protonate is a main mode and the others are under More modes:

Task	Command option	Example configuration
Set the salt type and concentration (NaCl, KCl, CaCl2 …)	`--mode salt`	`salt.yaml`
Change protonation states (HSD/HSE/HSP, ASPP, GLUP, LSN, CYM)	`--mode protonate`	`protonate.yaml`
Point mutations with a CHARMM36 topology and a clash-free rotamer	`--mode mutate`	`mutate.yaml`
Hydrogen mass repartitioning for 4 fs time steps	`--mode hmr`	`hmr.yaml`
Make the water layer thicker	`--mode resize`	`resize.yaml`
Add many copies of a small molecule to the water (ligand flooding)	`--mode flood`	`flood.yaml`
Add copies of the membrane protein (dimers, oligomers)	`--mode oligomer`	`oligomer.yaml`

After every run, the built system is checked with gmx grompp (no warnings allowed) when GROMACS is installed; gmxtransplant check-gromacs FOLDER checks any folder, gmxtransplant slurm-script writes a Slurm job for an equilibration folder, and gmxtransplant membrane-patch cuts the membrane reference to the host box and sets up its re-equilibration, so the new membrane fills the box with nothing lost at the edges (guide (docs/guides/MEMBRANE.md)).

Protein mode supports membrane proteins generally, including receptors, channels, and transporters. Any ligands included in the incoming selection move with the protein during alignment.

Install

Use Python 3.10 or newer. On Linux or WSL 2 with WSLg, clone the source:

```
git clone https://github.com/MidhunkMadhu/GMXtransplant.git
```

```
cd GMXtransplant
```

After cloning, choose one installation option below. Each block installs and starts GMXtransplant; run only one block.

Option 1: Python venv

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
gmxtransplant-gui
```

Option 2: Conda

Use Conda to create a Python environment, then install GMXtransplant from the cloned source:

```
conda create -n gmxtransplant python=3.12 pip
conda activate gmxtransplant
python -m pip install .
gmxtransplant-gui
```

On macOS, use the same clone and setup commands in Terminal. The command-line tool is gmxtransplant; run gmxtransplant --help to see its options.

If you already have a suitable Python environment, activate it and run python -m pip install . from the cloned source; creating another environment is optional. If you downloaded a source archive instead of cloning, open its extracted GMXtransplant directory and use the same installation options.

What is installed, and why

Package	Why GMXtransplant needs it
NumPy, SciPy	Coordinate arithmetic, rigid-body fitting and distance searches
PyYAML	Reading and writing the YAML configurations
MDAnalysis	Reading and writing GRO/PDB/MOL2 structures and periodic-box geometry
PySide6 (Qt)	The desktop application's windows. The command-line tool never loads it, so it also works on servers without a display
openbabel-wheel (Open Babel)	The obabel command that adds hydrogens and CHARMM36 names to experimental cholesterol (cholesterol restoration with convert_to_charmm36: true). Installed automatically on Linux and on macOS 14 or newer

Optional, installed separately

Package	Needed for	How to install
OpenMM	Running a prepared minimization folder	python3 -m pip install ".[minimize]" or conda install -c conda-forge openmm
PyMOL (with pymol2)	The pymol_align / pymol_cealign alignment methods; the default mask_fit does not need it	conda install -c conda-forge pymol-open-source
VMD	The Open in VMD comparison view	From the VMD website

On macOS, PyMOL and VMD installed as applications in /Applications are found automatically. For a copy elsewhere, set GMXTRANSPLANT_PYMOL or GMXTRANSPLANT_VMD to its full path.

If a dependency cannot be installed

- Qt on older Linux systems or some clusters: install just the command-line tool with GMXTRANSPLANT_CLI_ONLY=1 python3 -m pip install .. Every mode, example and option remains available through gmxtransplant; the application can be added later with python3 -m pip install "PySide6>=6.6,<7".
- Open Babel on Linux or WSL: if the pip package fails, install GMXtransplant with GMXTRANSPLANT_NO_OPENBABEL=1 python3 -m pip install ., then install Open Babel with your package manager, such as sudo apt-get install openbabel.

- Open Babel on macOS 12 or 13: no ready-built pip package exists, so it is skipped. Install it with brew install open-babel or conda install -c conda-forge openbabel.

Only cholesterol restoration needs Open Babel; check with obabel -V.

For an editable installation, use python3 -m pip install -e .. To run the command line from source without installing the package, use python3 src/run_pipeline.py in place of gmxtransplant.

Desktop application

Launch the application from any directory:

```
gmxtransplant-gui
# Equivalent module entry point:
python -m gmxtransplant.gui
```

The desktop application supports Linux, WSL 2 with WSLg, and macOS. It includes an input editor for all configuration sections, file and folder browsing, one-click examples, validation, readable progress summaries, cancellation, and result browsing. Detailed logs are collapsed until View Command Progress is clicked. YAML is generated automatically in the background; there is no YAML editing tab. Select an output folder first. Runs write directly to a folder named after the mode (charmprot/, protein/, ligand/, cholesterol/, addbinder/, membrane/, minimization/, and salt/, mutate/ … for the further modes); example runs use examples/<mode>/. Repeating a run replaces its generated products, including toppar/, without timestamped run directories.

Complete example inputs are included in the package for offline use. Read the GUI guide (docs/guides/GUI_USAGE.md) for a first run, input setup, results, and platform help.

The macOS installation has been tested on a MacBook Air.

Use a native Python installation matching your Mac's architecture. See the platform help (docs/guides/GUI_USAGE.md#platform-help) if the window does not open or the cholesterol workflow reports that Open Babel is missing.

Quick start

In the desktop application

1. Run gmxtransplant-gui and choose an output folder at the top of the window.
2. Open the Examples tab and click Run example on any card to see a complete run, or choose a mode on the Configuration tab and browse for your own inputs.
3. Click Check configuration, then Run pipeline, and follow the run on Run & results.

On the command line

Work in a separate directory and generate the example for your task:

```
mkdir my_gmxtransplant_job
cd my_gmxtransplant_job
gmxtransplant --show-example protein > protein_replace.yaml
```

Edit the paths, residue selections, topology sources, and scientific settings. Then validate the configuration and run assembly:

```
gmxtransplant --mode protein -i protein_replace.yaml --dry-run
gmxtransplant --mode protein -i protein_replace.yaml --output results/
```

--dry-run checks the configuration without opening molecular input files. Input identity, topology, and geometry checks happen during the normal run.

--output DIR chooses where the generated files go. Without it they are written to the directory you ran the

command from. Either way, the run's closing line names the absolute folder it wrote to.

Use --show-example with charmprot, lig, chl, addbinder, membrane, minimize, salt, protonate, mutate, hmr, resize, flood or oligomer for the other modes. Add --no-comments to print a compact example with the same active settings.

The source checkout includes a complete example for each mode. Each example README explains its supplied files and how to run it:

- Two-folder CHARMM-GUI transplantation (examples/charmprot/README.md)
- Protein insertion (examples/protein_insertion/README.md)
- Ligand replacement (examples/ligand_replacement/README.md)
- Cholesterol restoration (examples/cholesterol_restoration/README.md)
- Adding a binder (examples/addbinder/README.md): a prepared receptor system, a dopamine ligand and a G protein, placed on opposite sides of the membrane (guide (docs/guides/ADDBINDER.md))
- Equilibrated POPC membrane (examples/membrane/README.md): the same receptor system with its membrane replaced by pure POPC, and by a POPC/cholesterol bilayer from a GRO file (guide (docs/guides/MEMBRANE.md))
- The further modes, each on the same receptor system (guide (docs/guides/EDIT_MODES.md)): salt (examples/salt/README.md), protonate (examples/protonate/README.md), mutate (examples/mutate/README.md), hmr (examples/hmr/README.md), resize (examples/resize/README.md), flood (examples/flood/README.md), oligomer (examples/oligomer/README.md)

Prepare your inputs

The target environment can come from CHARMM-GUI, another system builder, or an MD trajectory frame. Supply matching coordinate and GROMACS topology files, including all force-field and molecule definitions. Incoming proteins and ligands also need matching parameters from a compatible force field.

GRO is preferred: it preserves the periodic box and four-character residue names such as POPC and TIP3. If using an MD frame, preserve atom order, whole molecules, and the correct box. Validated periodic processing requires an axis-aligned orthorhombic cell.

Example files group paths under paths: and reuse them as "${name}". Unknown or duplicate YAML keys are rejected.

Every input path must be written out in full. No input is resolved against the working directory, against the configuration file's folder, or against any shared input root, and a relative input path is rejected with a message naming the offending key. The one exception is the examples shipped with the package: their paths resolve against the examples/ folder installed alongside the code, so they run anywhere without referring to your filesystem.

Generated files are the mirror image: an output name such as step5_input.gro is a plain filename that lands in the run's output folder. That folder is --output DIR when you pass it, and otherwise the directory you ran the command from. Every run ends by printing the absolute path it wrote to.

Protein insertion

Set these together in protein_replace.yaml:

- target_box.protein_mask: the complete old protein/ligand block to remove.
- replacement_structure.protein_mask: the complete incoming block to insert, including any bound ligands.
- alignment: corresponding protein atoms used for fitting.
- replacement_ligands: coordinate residue names and ITP files for incoming ligands; charge and molecule type are read automatically from the ITP. Use [] when there are none. Each ligand may also give forcefield_path (see below).

- topology.protein_toppar_dir and topology.environment_toppar_dir: the protein and retained environment parameter sources.

The alignment transform is applied to the whole incoming block. Fit a stable protein region so that a changed ligand pose does not bias placement. For example, a receptor complex can be aligned using corresponding receptor backbone atoms.

mask_fit requires matching atom-name order and residue grouping, equal atom counts, and at least three non-collinear points. Use PyMOL alignment when correspondence must be established between different protein sequences.

Ligand replacement

In ligand_replace.yaml, select the prepared system and the original ligand residue. Use RESNAME:N to select a specific occurrence when its name appears more than once.

The incoming ligand needs three files: its coordinates (new_ligand.coord_path, e.g. a .mol2), its ITP (new_ligand.itp_path), and the force field holding its bond, angle and dihedral parameters (new_ligand.forcefield_path). A CHARMM-GUI ligand ITP only lists which atoms are bonded; the parameters are in the forcefield.itp CHARMM-GUI generated together with it. When forcefield_path is empty, a forcefield.itp next to the ITP is used, otherwise the environment's. Before writing the topology the run checks that every bonded term of the new ligand has parameters, and stops naming any that are missing.

Placement	Behavior
autofit	Fit matching, uniquely named heavy atoms
pairfit	Fit explicit pairs of atom names
mcsfit	Detect the largest shared substructure automatically (same elements and bonds), fit on its closest symmetric match, and report the pairs
nofit	Keep incoming coordinates that are already in the target frame

The protein is retained and protected from clash-based removal. Protein-ligand contacts are reported for inspection.

Cholesterol restoration

In cholesterol_restore.yaml, the experimental protein is aligned to the target protein and the associated cholesterol coordinates are transferred.

Optional CHARMM36 conversion regenerates hydrogens with Open Babel and validates the result against the included cholesterol reference. The example explains the limits for reconstructing missing heavy atoms.

Leaflet composition settings can remove surplus molecules outside the protected region around the protein. They do not create missing lipids or solvent.

Repacking before MD (repack/)

Issue: inserting the experimental cholesterols removes the lipids that overlap them, and the composition step then trims lipids far from the protein. The restored cholesterols start with few lipid neighbours (0–1 lipid heavy atoms within 5 Å instead of about 45). Released straight into MD they left their experimental poses within 25–50 ps in a D1R test, while the membrane shrank by 20%.

What is written: every run also writes repack/, a system set up for a short GROMACS equilibration (about 1.4 ns) that holds every heavy atom of the protein, the ligands and the restored cholesterols, and leaves all other lipids, water and ions free to fill the gaps. It has its own topol.top, toppar/, step5_input.gro, index.ndx and mdp files. A sample run script (sample_run.sh) is also saved. Afterwards, continue from the last GRO with the main topol.top.

Stage	Length	Protein, ligands, restored cholesterols (kJ/mol/nm²)	Other lipids
step6.0 minimization	5000 steps	4000	free
step6.1 NVT	125 ps	4000	free
step6.2 NVT	125 ps	2000	free
step6.3 NPT	125 ps	1000	free

```
step6.4 NPT             1 ns          1000                                                free
```

The restored cholesterols are their own moleculetype there (toppar/CHLR.itp: CHL1 with all heavy atoms restrained in x, y, z); protein and ligands use their CHARMM-GUI POSRES blocks, and the run warns if a heavy atom has none. In a D1R test with the protein and restored cholesterols held for 1.4 ns, the lipid heavy atoms within 5 Å of each restored cholesterol rose from 0–1 to 35–56 (normal: 59–73), the gaps closed within about 1 ns, and the poses held. The membrane area keeps settling for 10–15 ns afterwards, so release the remaining restraints gradually. Change the stages, restraints or temperature (310 K) under cholesterol.repack, or set enabled: false.

This restoration option supports cholesterol only. There is currently no corresponding option to preserve and transfer other experimentally resolved bound lipids. Existing lipids in the target membrane are retained according to the clash and composition settings.

Residue selections

Masks use one-based sequential residue positions in coordinate-file order, not the residue numbers printed in PDB or GRO files.

Mask	Selection
`:1-298`	All atoms in residues 1 through 298
`:1,5,10-20`	Selected residues or ranges
`:1-298@CA`	CA atoms in those residues
`:1-298@C,CA,N,O`	Backbone atoms in those residues
`:1-298&!:297-298`	Exclude residues 297 and 298

Optional OpenMM minimization

Prepare one openmm_minimization/ folder for a short clash-relaxation step. Preparation does not run or submit minimization. Execute the generated runner separately when ready.

Harmonic positional restraints keep protein and ligand heavy atoms and lipid head groups (the polar P, O, N and S atoms of lipids and sterols) near their starting coordinates. Lipid tails, hydrogens, water, and ions relax freely. These restraints allow small movements; they do not freeze the protein, ligand or membrane.

Prepare the folder

Use case	Command
Assemble and then prepare inputs from that run	`gmxtransplant --mode protein -i protein_replace.yaml --prepare-minimization`
Prepare inputs from an existing matching system	`gmxtransplant prepare-minimization --coordinates step5_input.gro --topology topol.top`

--prepare-minimization is an assembly option. It works with the protein, ligand, cholesterol and charmprot modes, and with addbinder, where it writes one folder per built pose (the binder's heavy atoms are restrained like the protein's). It requires topology.enabled: true and uses the freshly generated GRO and topology. Setting minimization.enabled: true in the assembly YAML has the same effect.

prepare-minimization is a separate command for existing files and does not repeat assembly. Use --include-dir for an additional topology search directory and --output to choose a different destination.

For adjustable standalone settings:

```
gmxtransplant --show-example minimize > minimization.yaml
# Edit the input paths and settings.
gmxtransplant prepare-minimization -i minimization.yaml
```

The folder contains coordinate and topology copies, all topology dependencies, an expanded topology for execution, a settings YAML, the Python runner and its support files, run.sh, generic multicore CPU and GPU SLURM templates, a README, and an input audit with checksums. Move the complete folder to your execution machine; GMXtransplant itself is not required there.

Settings

Every setting has a default suited to CHARMM36 membrane systems, so an assembly configuration needs only minimization: {enabled: true}. The settings you are most likely to change are:

```
minimization:
  enabled: true
  restraint_force_constant_kj_mol_nm2: 1000.0
  max_iterations: 5000
  platform: auto            # or CPU, CUDA, HIP, OpenCL
  restraint_residue_classes: {}   # e.g. {MYLIP: lipid} for a custom lipid
```

For nonbonded, constraint, and SLURM resources settings, edit the YAML file.

Restraints use the input coordinates as references and a default force constant of 1000 kJ/mol/nm². Protein molecules are recognized from their amino-acid residues, including attached caps and modifications. Unrecognized molecules are treated as ligands. Classify custom lipids and unusual residues explicitly, for example restraint_residue_classes: {MYLIP: lipid}. Supported classes are protein, ligand, lipid, water, and ion.

The runner uses OpenMM L-BFGS with a finite iteration limit and an RMS objective-gradient tolerance. Review the example's nonbonded settings for your force field. OpenMM potential switching differs from LJ force switching; this is a configurable clash-relaxation step, not a universal simulation protocol.

Topology position_restraints must be inactive because the runner adds its own restraints. Preparation rejects active definitions instead of ignoring them.

Run separately

From the generated folder:

```
cd openmm_minimization
python3 -m pip install -r requirements.txt
bash run.sh
```

The default run uses multiple CPU threads. Set OPENMM_CPU_THREADS to adjust the local thread count, or select a GPU with bash run.sh --platform CUDA. HIP and OpenCL can also be selected when available.

For SLURM, review the resource directives and software setup, then submit one template from this folder:

```
sbatch submit_cpu.slurm
# Alternatively:
sbatch submit_gpu.slurm
```

The CPU template uses one process with multiple threads. The GPU template requests one GPU and selects CUDA. An unavailable requested platform produces an error.

Inspect results

The runner writes:

- results/minimized.gro
- results/minimization_report.txt
- results/minimization_report.json

Reports include convergence, energies, forces, selected restraint counts, inferred ligand names, and heavy-atom displacement. Energies include the positional restraints. Exit code 7 means failure or unconfirmed convergence; inspect the report before using any output coordinates.

Existing preparation and results directories are never overwritten. Choose a new destination for preparation, or preserve an earlier results/ directory by renaming it before rerunning. Minimization does not replace equilibration.

Assembly checks and outputs

Assembly checks the periodic environment, aligns the incoming component, handles clashes, optionally corrects charge, and rebuilds the topology molecule list.

Clash cutoffs act on complete environment residues. Every mode uses the same clash_detection.threshold of 0.80 Å by default, and no molecule class carries an implicit cutoff of its own: cholesterol is judged exactly like any other lipid. Override a single class through clash_detection.thresholds when a system needs it. Classes in clash_detection.keep_classes are retained and their contacts are reported. Neutralization removes eligible existing counterions; it does not add ions or set a new salt concentration.

Cholesterol is treated as a distinct molecule only in cholesterol-restoration mode, which is the mode that reasons about it. In protein, ligand and CHARMM-GUI transplant mode it is classified, coloured and reported as an ordinary membrane lipid.

A successful run produces GRO/PDB coordinates, an inspection PDB, text/JSON reports and index.ndx, plus topol.top and toppar/ when topology writing is on (as in the examples). Review the alignment, removed molecules, retained clashes, lipid counts, net charge, and coordinate/topology checks before continuing.

The commented example files describe box validation, name restoration for truncated PDB names, clash thresholds, ion protection, and output options.

Limitations

- All components must already have compatible parameters. GMXtransplant does not parameterize new molecules.
- Validated periodic processing supports orthorhombic boxes. Use GRO as the authoritative simulation coordinate output.
- Experimental bound-lipid restoration supports cholesterol only; other bound lipids have no dedicated preservation/restoration option.
- Composition correction removes surplus lipids but does not build missing lipids or solvent.
- Conditional topology definitions may need preprocessing. Minimization preparation supports literal includes, object macros, and #ifdef/#ifndef branches; unsupported expressions and cyclic includes are rejected.
- Preparation checks input consistency without importing OpenMM. OpenMM parsing and force-field compatibility are checked when the runner executes.
- Exact overlaps, poor placement, or incorrect parameters may need correction before minimization can succeed.

Runnable protein insertion example

The repository includes a complete input system in examples/protein_insertion (examples/protein_insertion/README.md). Its relative paths are resolved against the installed example folder, so it runs from anywhere:

```
python -m pip install .
gmxtransplant --mode protein \
  -i "$(python3 -c 'from gmxtransplant.gui.model import example_path; print(example_path("protein"))')" \
  --output ~/gmxtransplant-example
```

environment/ supplies the target box, membrane, solvent and ions. replacement/ supplies the selected incoming protein and bound ligands only. Protein topology definitions come from the replacement; retained environment molecules use environment definitions. Explicit incoming ligand ITPs supersede old copies in either directory. Conflicts within a source remain errors.

In the generated examples, incoming ligand names and ITP paths are together near the top. charge: from_itp and moleculetype need not be repeated. Coordinate resname and ITP [moleculetype] names need not be

identical: the unique matching [atoms] residue name determines the type. Ambiguous ITPs still require an explicit molecule type. Detailed YAMLs can set these fields explicitly.

For protein-mode PDB name restoration, the environment's topol.top is found beside environment_toppar_dir; no separate reference topology is needed. A nonstandard layout can still set name_restoration.reference_topol explicitly. The example enables name restoration so truncated POP/TIP/CHL names are classified correctly before clash removal. Set masks, cutoffs, protection, and composition explicitly in YAML. Standalone minimization settings are in minimization.yaml.

Development

Directory	Contents
src/	Python pipeline and desktop application
configs/	Configuration templates
examples/	Complete example inputs and configurations
docs/guides/	User guides
docs/images/	Guide screenshots
tests/	Regression tests
tools/	Documentation builder

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
python3 -m pip install build
python3 -m build
```

GitHub Actions tests Python 3.10, 3.11, and 3.12, builds the package, and runs OpenMM tests with the optional dependency installed.

Desktop application

GMXtransplant helps you build a new molecular system from prepared structures. You choose the inputs, check the setup, run the calculation, and inspect the results in one desktop window. Your input files stay on your computer.

Open the application

If GMXtransplant is already installed, open a terminal and run:

```
gmxtransplant-gui
```

To install on Linux or WSL 2 with WSLg, first clone the source:

```
git clone https://github.com/MidhunkMadhu/GMXtransplant.git
cd GMXtransplant
```

Choose one of these options; run only one block.

Option 1: Python venv

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
gmxtransplant-gui
```

Option 2: Conda

```
conda create -n gmxtransplant python=3.12 pip
conda activate gmxtransplant
python -m pip install .
gmxtransplant-gui
```

If you already have a suitable Python environment, activate it and run python -m pip install . from the cloned folder. On macOS, use the same commands in Terminal.

The next time you open a terminal, activate the environment you chose, then run gmxtransplant-gui. On Linux, use a graphical desktop session. On Windows, use WSL 2 with WSLg. On macOS, open the application from a graphical desktop session. See platform help (#platform-help) if the window does not open.

Try an included example

The examples include the input structures and settings they need. They are a good way to learn what a completed run looks like.

1. At the top of the window, click Choose folder… and select a folder for results. The box initially shows the folder from which you launched the application; check it before running.
2. Open the Examples tab. Choose an example card and click Run example. The add binder cards demonstrate a ligand above a receptor and a G protein below it. Other cards cover protein transplant, protein insertion, ligand replacement, cholesterol restoration, the POPC membrane and protonation states. Click More examples for a POPC/cholesterol membrane, salt, mutations, hydrogen mass repartitioning, a thicker water layer, ligand flooding and a receptor dimer.
3. Follow the run in Run & results. Progress at a glance shows the current step and key findings. Click View Command Progress for the full log if you want more detail.
4. When the run finishes, the result folder appears above the file list. Double-click a report to open it. If PyMOL or VMD is installed, use its button to open the generated view.

The Examples tab

To use an example as a starting point for your own system, click Open in editor. The form will open with its

settings filled in. An example run saves to an examples/ folder; a run started from the editor saves to the selected mode's folder.

Set up your own system

Open the Configuration tab and choose the task that matches your goal. The main modes are always shown; click More modes for the modes that change one prepared system (guide (EDIT_MODES.md)):

Task	What it does	Main inputs
CHARMM-GUI protein transplant	Keeps the membrane, water, and ions from one prepared system, then replaces its protein and ligands with those from another.	Reference and transplant system folders
Protein insertion	Replaces a protein in a prepared system with a new protein and any bound ligands.	Target and replacement structures with matching topology files
Ligand replacement	Replaces one bound ligand while keeping the rest of the system.	Prepared system, incoming ligand structure, GROMACS ITP parameter file, and force field
Cholesterol restoration	Places experimentally resolved cholesterol into a prepared system.	Target system and experimental structure
Binder placement	Places a ligand or protein above or below a membrane protein, with a separate system for each pose.	Prepared host system, binder coordinates, and binder parameters
Membrane replacement	Replaces the membrane with equilibrated POPC, or any bilayer GRO you give, keeping the OPM placement; writes a GROMACS equilibration folder.	Prepared host system; optionally a bilayer GRO
Protonation states	Changes residue protonation states (HSD/HSE/HSP, ASPP, GLUP, LSN, CYM) and rebuilds the topology.	Prepared host system and the residues to change
Minimization preparation	Creates a folder of inputs for a restrained OpenMM minimization.	Assembled coordinates and topology

For add binder inputs and placement options, see the addbinder guide (ADDBINDER.md). The included examples (../../examples/addbinder/README.md) show a ligand above a receptor and a G protein below it. For membrane replacement, see the membrane guide (MEMBRANE.md).

Fill in the visible input fields first. Click Browse… or Folder… to select each file or folder. If you type a path, enter its full location. Open Advanced settings when you need to change selections, geometry, topology, composition, or other scientific settings. Help text appears beside each setting. You can switch between tasks without losing the edits you made to each task during this session.

The Configuration tab

Before starting a full run:

1. Click Check configuration to find missing or invalid settings.
2. Click Check input paths to check that the named input files and folders can be used.
3. Click Run pipeline. The full run also checks the assembled system and reports problems it finds. The two earlier checks do not build a system.

If a check fails, read the message at the top of the window. The Run & results tab has more detail under View Command Progress. Correct the named field and run the check again.

Find and review your results

GMXtransplant creates a subfolder inside the output folder you selected. Each task has its own subfolder, such as protein/, ligand/, cholesterol/, addbinder/, or charmprot/. Included examples go under examples/. An add binder run also creates one folder per pose.

Look for the report and the final coordinate and topology files in that subfolder. run.log contains the detailed messages. run-status.json records whether the run completed, failed, or was cancelled. run.yaml saves the settings and input locations used for that run, so you can keep a record of how the system was built.

Running the same task again in the same output folder replaces files from the previous GUI run. Choose a different output folder if you want to keep both sets of results. Your source inputs are left in place. If an existing

file has a name GMXtransplant needs but was not made by an earlier GUI run, the application stops before replacing it. Move that file or choose another output folder.

Click Cancel run to stop a run. A failed or cancelled run may leave partial results in its folder; check the status and log before using those files.

PyMOL and VMD are optional. Their buttons appear only when the application finds them. See the visualization guide (VISUALIZATION.md) for the scene colors and the structures shown together.

Common questions

Why does an input path fail? Use Browse… or Folder… to select the actual input. A typed path must include its full location. Choosing an output folder does not set a base folder for inputs.

Why is a setting missing from the form? Look in Advanced settings on the relevant page. Some settings become visible after you turn on their section.

Where did the output go? Open Run & results. The full result folder is shown above the file list after a run. It is also named at the end of the log.

Why does the cholesterol example mention Open Babel? That workflow needs Open Babel to prepare cholesterol coordinates. If the application reports it missing, install Open Babel and restart GMXtransplant.

Where is the full manual? Click Documentation in the left panel to open the manual included with the application.

You can change the appearance with Theme at the top right. Your choice is remembered the next time you open the application.

Platform help

On Linux, launch the application from a graphical desktop session. If you are connected to a remote computer without a desktop, use the command-line tool described in the README (../../README.md).

On Windows, use WSL 2 with WSLg and install GMXtransplant inside the Linux environment. Browse using Linux paths, including /mnt/c/ for the Windows C: drive. If the application window does not appear, check that graphical Linux applications work in your WSL installation.

On macOS, install in a Python environment that matches your Mac. The cholesterol workflow also needs Open Babel. Install it separately if GMXtransplant reports that it is missing.

CHARMM-GUI transplant mode

Run the installed package:

```
gmxtransplant --mode charmprot -i charmprot.yaml --dry-run
gmxtransplant --mode charmprot -i charmprot.yaml
```

Or from the source checkout:

```
python -B src/run_pipeline.py --mode charmprot -i charmprot.yaml --dry-run
python -B src/run_pipeline.py --mode charmprot -i charmprot.yaml
```

The entire required YAML is:

```
charmprot:
  reference: /path/to/reference/gromacs
  transplant: /path/to/donor/gromacs
```

Each folder must contain topol.top, toppar/, and step5_input.gro (or exactly one step5*.gro). Both folder paths, and any reference_gro/transplant_gro override, must be written out in full: nothing is resolved against the YAML's directory or the working directory. Inputs are read-only. output_dir defaults to the name charmprot_output, placed in the --output directory when you pass one and otherwise in the directory you ran the command from. Rewriting an existing output_dir is the norm: generated artifacts are replaced transactionally, with rollback if publication fails. Unrelated files and GUI logs/locks are preserved. A failed assembly publishes failure reports and removes stale generated results. --dry-run validates the YAML without opening the input folders.

Detection and selection

The program reads the actual topology include graph and expands [molecules] using ITP atom counts. Every coordinate atom must agree with its ITP residue and atom name in file order. GRO serial-number wrapping does not affect selection. Unreferenced ITP files do not affect classification.

PROA, PROB, and other PRO... topology molecule types identify proteins. An additional backbone check recognizes protein types with other names. Every detected protein is selected by default. A covalent cofactor inside the same protein moleculetype travels with that molecule; ligands: ignore only applies to separate ligand molecules of the transplant.

Environment categories are water, ion, lipid, sterol, detergent and solvent. The bundled catalogue is a dated snapshot of 943 distinct identifiers from 19 primary-source pages, supplemented with exported water/ion/lipid aliases. It includes every download identifier on the retrieved Individual Lipid Library page and every RESI identifier in the retrieved lipid, water/ion and noble-gas topology files linked from the CHARMM-GUI topology index. It does not treat patch identifiers (PRES) as complete molecules.

Primary sources:

- CHARMM-GUI Individual Lipid Library (https://www.charmm-gui.org/?doc=archive&lib=lipid)
- CHARMM-GUI topology index (https://www.charmm-gui.org/?doc=toppar)
- Water and ions topology (https://www.charmm-gui.org/?doc=open_toppar&filename=toppar_water_ions.str)
- Extended ion topology (https://www.charmm-gui.org/?doc=open_toppar&filename=stream/misc/toppar_ions_won.str)
- CHARMM-GUI water-model FAQ (https://www.charmm-gui.org/?doc=faq)
- OpenMM CHARMM water distribution (https://github.com/openmm/openmmforcefields/blob/main/charmm/files/waters.yaml)

The catalogue records a retrieval date, source hashes and per-name source references in gmxtransplant/charmm_environment.json. charmm_environment.py contains export aliases. Ordinary runs

work entirely offline.

Names are matched exactly, case-insensitively. Both molecule names and the ITP residue names are examined. Thus all catalogued POP... lipids are covered without an unrestricted POP* rule. Single charged atoms also qualify as ions even when their name is custom. Unknown non-protein molecules become inferred ligands and are named explicitly in the report. Topology position after the protein and before the environment is recorded as supporting evidence, not proof of chemical identity.

No fixed name list can guarantee every future, custom or renamed CHARMM-GUI component. The snapshot's coverage is stated precisely above; unfamiliar names remain visible in the report. Use an explicit ligand list or environment override when a name has a different biological role (e.g. a bound metal or functional cholesterol molecule).

Optional YAML

```
charmprot:
  reference: /path/to/reference/gromacs
  transplant: /path/to/donor/gromacs
  reference_proteins: [PROA, PROB]
  transplant_proteins: [PROB, PROA]
  reference_ligands: [LI1]
  transplant_ligands: [LI2]
  output_dir: transplant_result
```

Protein lists specify which complete molecule types to replace and the chain pairing order. Multiple instances of a selected type are all selected, in file order. Omitted protein types in the reference remain in place. Ligand lists take precedence over the environment catalogue; [] explicitly selects no ligands on that side. Omission means automatic detection on that side.

ligands chooses what is inserted from the transplant:

- auto (default): the reference's proteins and ligands are removed, and the transplant's proteins and ligands are inserted.
- ignore: the reference's proteins and ligands are removed in the same way, but only the transplant's proteins are inserted; its ligands are left out. reference_ligands may still name which reference ligands to remove; transplant_ligands cannot be combined with ignore.

If a retained protein or ligand (one not selected for removal) clashes with the incoming block, the run stops and reports the conflict.

Additional options:

Option	Default	Meaning
reference_gro, transplant_gro	automatic	Full path to a coordinate file, when the folder's own step5*.gro is not the one to use
environment_overrides	{}	Explicit molecule-name to category mapping, e.g. {MYLIP: lipid}
clash_distance	0.80	Heavy-atom environment clash distance in Å; removes complete molecules
keep_lipids	false	Keep every lipid and sterol, even when it clashes; only other clashing molecules (water, ions, solvent) are removed. Retained clashes are listed in the report; minimize before simulating
neutralize	true	Remove eligible counterions to restore target charge
target_net_charge	0.0	Desired charge in elementary-charge units
ion_exclusion_distance	10.0	Minimum ion distance from inserted/retained biological molecules in Å
lipid_ion_exclusion_distance	5.0	Minimum ion distance from retained membrane heavy atoms in Å

Alignment, topology and outputs

Corresponding protein chains are globally aligned by residue sequence, followed by a single Calpha Kabsch rigid fit over all matched chains. This supports missing residues and substitutions without PyMOL. Chain counts must agree; low-identity pairings stop with a clear error. Symmetric chain permutations are not searched automatically: give corresponding protein lists if file order differs. All selected donor atoms, including ligands, receive exactly the same transform. The donor protein should be whole under periodic boundaries

before alignment.

Environment clashes use minimum-image distances with the reference box. Whole topology molecules are removed, including multi-residue lipids. A 0.02 Å margin allows for GRO rounding. Exact molecule charges come from ITP [atoms] records. Charge balancing removes distant counterions with the needed charge; it stops if the requested charge cannot be reached. It does not add ions.

The parameter sets are combined, identical definitions are deduplicated, and conflicting definitions stop the run. Protein and ligand molecule definitions come from the donor; retained molecules keep reference definitions. Conditional include graphs, parameter macros and nested molecule includes need preprocessing; unsupported ambiguity is rejected. Standard flat CHARMM-GUI toppar inputs are the intended format.

On success the output folder contains:

- step5_input.gro, step5_input.pdb: combined coordinates in the reference box
- topol.top, toppar/: assembled parameters and molecule definitions
- index.ndx: topology-based protein, ligand, environment and component groups
- aligned_inspection.pdb: reference and transformed donor selection overlay
- charmprot_report.txt, charmprot_report.json: both systems' classifications, selection decisions, unknown names, alignment, molecule removals and charge audit

The final topology is reparsed independently and checked against the written GRO atom order and charge. Final files are published together from temporary storage. On failure only diagnostic reports are published, with status: failed. The reports are the record of exactly what was considered environment and ligand.

--prepare-minimization also exports a portable minimization bundle after a successful transplant; it does not run minimization.

GUI, examples and visualization

In the desktop application, select CHARMM-GUI protein transplant and choose the Reference and Transplant folders. Select the output folder at the top of the window. Other settings are under Advanced settings. The Examples card runs the supplied datasets in examples/charmprot/.

Canonical configurations use a charmprot: section and may use the shared paths: registry. The original flat two-folder YAML is still accepted. A root minimization: section uses the same settings as other modes; --prepare-minimization is also supported.

All successful modes produce view.pml, view.vmd, and a visualization/ asset directory. A view.pse session is also created when PyMOL is installed. See Visualization (VISUALIZATION.md) for colors and input overlays.

Add binder mode

--mode addbinder takes a prepared membrane system and adds one incoming molecule (a ligand or a protein) in the water above or below the membrane protein, at a set distance from the protein's outermost heavy atom. Typical uses:

- an apo receptor with its ligand placed outside the pocket, ready for unbiased binding simulations
- a partner protein placed near the receptor's extracellular or intracellular face

```
gmxtransplant --show-example addbinder > addbinder.yaml   # edit the paths
gmxtransplant --mode addbinder -i addbinder.yaml --dry-run
gmxtransplant --mode addbinder -i addbinder.yaml --output results/
```

To try the included dopamine and G-protein examples, see the addbinder example guide (../../examples/addbinder/README.md). The desktop application also has a card for each example on its Examples tab.

Inputs

Key	What to give
host	Folder holding the system the binder joins: topol.top, toppar/ and step5_input.gro (or one step5*.gro; host_gro overrides it). A CHARMM-GUI membrane build fits directly. The GRO must match the topology atom for atom.
binder_coordinates	PDB, GRO or MOL2 of the binder, hydrogens included. Atom names must match the ITP(s); for a single molecule with unique names the order may differ. For a multi-molecule binder, the molecules appear in binder_itp order. Position and orientation do not matter except for orientation: as_is.
binder_itp	The binder's ITP with exactly one [moleculetype], or a list of ITPs, one molecule each, for a binder made of several molecules (e.g. a G-protein heterotrimer [PROB.itp, PROC.itp, PROD.itp]). The molecules are placed as one rigid body. A name that clashes with a different host molecule type (CHARMM-GUI calls protein chains PROA, PROB, ...) is renamed <name>_BND.
binder_forcefield	The force field holding the binder's parameters, e.g. the forcefield.itp CHARMM-GUI Ligand Reader generated with the ITP. Parameters are merged with the host's; the same key with different values stops the run. Before building, every binder molecule (protein or ligand) is checked: each atom type must be in [atomtypes], and each bond, angle, dihedral and CMAP term must have parameters, in the host's force field plus this one. A gap stops the run and names the terms and residues (e.g. a non-standard residue). Note that a CHARMM-GUI forcefield.itp holds only what its own system uses, so a protein from a different build usually needs its own.

For your own configuration, give input paths in full. The supplied examples resolve their relative input paths against the example folder.

How placement works

1. Membrane frame. The two headgroup planes are the median z of the lipid phosphorus atoms in each leaflet. The bilayer must be flat and normal to z. side: upper is +z and side: lower is -z. A coordinate file does not say which side is extracellular, so you choose.
2. Tip. The host protein and its bound ligands are made whole across the periodic boundary. The tip is their outermost heavy atom on the chosen side. The report names it, e.g. PROA HSD159 NE2.
3. Pose. The binder is rotated (see below), centred over the tip in x/y (lateral_reference: host_center uses the protein's centre instead), shifted by lateral_offset, and moved along the normal until its nearest heavy atom is distance Å beyond the tip plane.
4. Checks (heavy atoms, all 26 periodic images). Any failure rejects the pose and explains why:
   - distance to the host >= distance
   - distance to any periodic image of the host >= min_image_gap; when the closest image is along z, the message says how much more box height is needed
   - distance to the binder's own images >= min_image_gap
   - distance to lipid heavy atoms >= min_membrane_gap
5. Water and ions. Waters and ions with any heavy atom within water_clash_distance (2.4 Å) of any binder

atom are removed. Protein, ligand and lipid molecules are never removed.

6. Salt and charge. The binder displaces water and the ions in it, and brings its own charge. The final ion counts are therefore set directly:
   - the host's salt concentration is measured as salt pairs × 55.51 / waters, using its main monovalent cation and anion (e.g. SOD/CLA);
   - on the remaining water, pairs = concentration × waters / 55.51 (rounded);
   - the counterion species gets pairs + |net charge of everything else|, so the system reaches target_net_charge; the other species gets pairs.

Extra ions are removed and missing ones replace bulk waters, chosen at random among molecules at least ion_exclusion_distance (10 Å) from the protein and binder, lipid_ion_exclusion_distance (5 Å) from lipids, and (for added ions) 5 Å from any other ion. By default this uses the whole box (ion_side: any): in a single-bilayer periodic box the two water layers are one connected layer, so the ions redistribute during equilibration. ion_side: binder_side uses the binder's side first. Set salt_concentration to a number (mol/L) to use a fixed concentration instead of the host's.

7. Output. Coordinates in topology order: host proteins, binder proteins, host ligands, binder ligands, environment. The binder ends up inside the box and whole. The written topology is re-read and audited against the coordinates, including the charge.

Orientation

orientation	Meaning
auto (default)	A different orientation for each pose, whether or not the poses share a distance.
flat	Principal axes: longest along x, thinnest along the membrane normal. This uses the least box height.
end_on	Longest axis along the normal. The end that reaches furthest from the centre (e.g. a ligand's tail rather than its ring) faces the protein.
edge	Middle axis along the normal, longest along x.
as_is	Keep the input orientation, e.g. a docked or pocket pose.
euler	Rotate by angles: [z, y, z] (degrees) about the binder centre.

flip: true turns the binder upside down (180° about x). spin (degrees) then turns it about the membrane normal.

auto poses take, in turn: flat, end_on, end_on flipped, edge, flat flipped, edge flipped. It skips any of these that a pose already uses explicitly. After six it starts again, turned 45° further about the normal. With same_orientation: true, every auto pose gets the first one (flat). Give a pose its own distance to vary the distance too.

Where each pose goes

Pose setting	Placement	distance means
(default)	straight out along the membrane normal, above (upper) or below (lower) the tip, shifted by lateral_offset	gap along the normal from the tip atom to the nearest binder heavy atom
approach: [tilt, azimuth]	out from the tip along a direction tilted tilt° from the normal towards azimuth° (0 = +x, 90 = +y)	gap from the nearest host heavy atom to the nearest binder heavy atom
centroid: [x, y, z]	binder heavy-atom centre exactly at x, y, z (Å, in the host GRO's coordinates, i.e. GRO nm × 10)	not used; the pose is rejected only if it overlaps the host (under 3 Å) or fails the box checks
distance_to: centroid (with optional approach)	the binder's heavy-atom centre at distance from the tip (or the host centre, lateral_reference: host_center) along the normal or the approach direction: spherical coordinates (r = distance, theta, phi = approach). Rotate with orientation: euler and angles: [z, y, z]	centre-to-reference distance; rejected only if it overlaps the host (under 3 Å) or fails the box checks

from_input_position: true	starts where the binder is in its input file (coordinates already in the host's frame, e.g. a G protein in its receptor-bound arrangement) and moves it straight out along the normal, or along approach	gap from the nearest host heavy atom to the nearest binder heavy atom

Every pose can have its own distance. As many poses as you like can be listed. With reduce_distance_by (e.g. 2) a pose that does not fit the box is retried at a shorter distance, down to min_distance (default 10 Å); the report lists every distance tried.

Random poses

After the listed poses, random_poses (default 3) more are drawn, named numbered on from the listed ones (binderpose2, binderpose3, ...). Each one gets:

- a random approach direction, up to random_max_tilt (60°) from the normal
- a random distance in random_distance (default: distance to distance + 5)
- a random orientation

A draw is thrown away and drawn again (up to 2000 times) when:

- it fails the box or membrane checks
- its direction is within random_min_angle (25°) of another pose's
- its binder centre is within random_min_separation (10 Å) of another pose's

The same random_seed gives the same poses. Each report records the values drawn, so a pose you like can be copied into poses as distance, orientation: euler, angles and approach. Set random_poses: 0 to place only the listed poses.

For a binder whose input is already a meaningful pose (e.g. a G protein in its receptor-bound arrangement), random_from_input_position: true moves the binder out from its input position along each random direction, and random_rotated: N gives only the last N random poses a random rotation; the others keep the input orientation, so only their side and tilt are random.

Choosing distances

The box repeats along z. Above the extracellular tip lies the intracellular side of the next periodic copy. The binder therefore needs:

```
distance + binder height along z + min_image_gap <= free water column
```

The free water column is the box height minus the solute's z extent. Each report gives free_water_column_angstrom and column_room_for_binder_angstrom. The real 3D check is usually more forgiving, because the two tips are rarely directly above each other. In the bundled D1R example the column is about 158 Å, so dopamine at 20 Å fits easily and so would a G protein.

Output

```
<output>/addbinder_output/
  summary.txt, summary.json      one row per pose: status, gaps, removed molecules
  <pose>/step5_input.gro/.pdb    full system with the binder
  <pose>/topol.top, toppar/      merged force field + one ITP per molecule type
  <pose>/index.ndx               System Protein Ligand Host Binder Host_tip Solute Water Ion Membrane Environment
                                 + CHARMM-GUI SOLU MEMB SOLV SOLU_MEMB SYSTEM
  <pose>/*.mdp, README           the host's CHARMM-GUI run inputs (copy_run_inputs)
  <pose>/binder_restraint.mdp    the binder restraint (restraint.enabled), also appended to the MD mdp files
  <pose>/openmm_minimization/    with --prepare-minimization: an OpenMM minimization folder per pose
  <pose>/addbinder_report.txt/.json
  view.pml, view.pse, view.vmd   one PyMOL/VMD scene: host, tip atom, every pose's binder
```

A rejected pose writes only its report. A pose folder is replaced on reruns only if it holds nothing but addbinder's own files.

Keeping the binder near the protein

Without a restraint, a free binder diffuses away. restraint: {enabled: true} writes binder_restraint.mdp: a flat-bottomed GROMACS pull restraint between the Host_tip atom (or, with reference: host, the host proteins' centre of mass) and the Binder centre of mass. There is no force up to the starting distance plus margin (5 Å); beyond it a harmonic wall of force_constant (1000 kJ/mol/nm²) pulls the binder back. With apply_to_run_inputs (default) it is appended to every copied MD mdp file (step6.1 to step7), not to the minimization. A GROMACS distance pull must stay below half the shortest box length; a pose farther away gets no restraint, and its report says so.

--prepare-minimization writes an OpenMM minimization folder in each built pose. Its restraints hold every protein and ligand heavy atom, the binder's included, so the binder stays in place while water, ions and lipid tails relax.

All options

Option	Default	Meaning
side	upper	upper (+z) or lower (-z)
distance	10.0	Å, default for poses without their own
min_image_gap	10.0	Å, to periodic images of host and binder
min_membrane_gap	5.0	Å, to lipid heavy atoms
lateral_reference	tip	tip or host_center
water_clash_distance	2.4	Å, binder atom <-> water/ion heavy atom
same_orientation	false	true: every auto pose gets the same orientation
copy_run_inputs	true	copy the host's *.mdp and README into each pose folder
random_poses	3	extra poses from random directions (0 for none)
random_distance	[distance, distance + 5]	Å, range for the random poses
random_max_tilt / random_min_angle	60 / 25	degrees
random_min_separation	10	Å between binder centres of any two poses
random_seed	1	same seed, same random poses
random_rotated / random_from_input_position	all / false	how many random poses are also rotated; start from the input position
salt_concentration	host	host (keep the host's) or mol/L
neutralize / target_net_charge	true / 0	add counterions to reach the target charge
ion_side	any	any (whole box) or binder_side (binder's side first)
ion_exclusion_distance / lipid_ion_exclusion_distance	10 / 5	Å, protected ions
environment_overrides	{}	molecule type -> water/ion/lipid/sterol/detergent/solvent
output_dir	addbinder_output	inside --output
restraint	disabled	enabled, reference (host_tip or host), margin (Å), force_constant (kJ/mol/nm²), apply_to_run_inputs
poses[]	one pose straight above the tip	name, distance, lateral_offset, orientation, angles, flip, spin, approach, centroid, from_input_position, distance_to, reduce_distance_by, min_distance

Desktop application

In the desktop application, choose Binder placement on the Configuration tab, or run one of its two cards on the Examples tab: dopamine (extracellular) or the Gs trimer (intracellular). Results go to addbinder/ (examples: examples/addbinder/ and examples/addbinder_gprotein/) in the output folder.

Limitations

These are not supported yet:

- automatic box extension (use the resize mode (EDIT_MODES.md#resize) on the host first)
- protein binders build (the Gs example) but have not yet been simulated

Membrane replacement

--mode membrane replaces the membrane of a prepared system with the lipids of a well-equilibrated bilayer: pure POPC from the bundled 500 ns bilayer by default, or any bilayer you give as a GRO file (mixed lipids, cholesterol, asymmetric leaflets). The protein and its ligands are not moved: the host membrane's midplane is found from its lipids and the new bilayer is centred on it, so the OPM placement that CHARMM-GUI built around is kept.

Use it to:

- start from lipids with equilibrated packing, tail order and headgroup hydration instead of a freshly built membrane
- turn a mixed membrane (e.g. POPC/POPE/cholesterol) into pure POPC, or into another composition from your own equilibrated bilayer

Quick start

```
gmxtransplant --show-example membrane > membrane.yaml   # set host to your system folder
gmxtransplant --mode membrane -i membrane.yaml --dry-run
gmxtransplant --mode membrane -i membrane.yaml --output results/
```

The only required setting is host. To try the included D1 receptor system first, see the membrane example (../../examples/membrane/README.md). It is also a card on the Examples tab of the desktop application.

Inputs

Key	What to give
host	Your system folder: topol.top, toppar/ and step5_input.gro (or one step5*.gro; host_gro picks another GRO, e.g. an MD frame). A CHARMM-GUI Membrane Builder folder fits directly. The host's own lipid ITPs (POPC.itp, CHL1.itp …) are used for the new lipids.
reference_gro	Optional. Leave it out for pure POPC from the bundled bilayer (below). Give any other equilibrated bilayer GRO in CHARMM36 names for another composition. Each residue is matched to a moleculetype by name, else to the host lipid or sterol with exactly the same atom names (CLOL becomes CHL1); the atom order may differ. Three-site water in it hydrates the new bilayer; ions in it are ignored.
lipid_itps, lipid_forcefield	Only for a reference with lipids the host does not have (e.g. POPS.itp), and their parameters when the host forcefield.itp lacks them. resname_map forces a residue -> moleculetype match.
membrane_center_z	Optional. The membrane midplane in Å, in the host GRO frame. Leave it out to use the host membrane's own midplane, the centre the host was built around.

A second example uses your-own-GRO input: examples/membrane/popc_cholesterol.yaml replaces the D1R membrane with an equilibrated POPC/cholesterol 70:30 bilayer (J. Melcr, Zenodo 10.5281/zenodo.209362 (https://doi.org/10.5281/zenodo.209362), CC BY 4.0).

The bundled bilayer

The default reference is the last frame of a 500 ns simulation: 512 POPC and 20,480 waters, CHARMM36, 150 mM NaCl, 310 K. It comes from H. Martinez-Seara, Zenodo 10.5281/zenodo.1010083 (https://doi.org/10.5281/zenodo.1010083), licensed CC BY 4.0. Only its ions were removed. It has a P–P thickness of 39.0 Å and 64.5 Å² per lipid. Cite the Zenodo record when you publish with it.

What happens

1. Find the host midplane. The midplane is found from the tail ends of the phospholipids (any of them), which gather at the bilayer centre; with fewer than six phospholipids, from all membrane heavy atoms. Each leaflet's phosphorus plane must be within 5° of the xy plane: the membrane must be flat and normal to z.

2. Remove the host membrane. Every lipid, sterol and detergent is removed. The protein, ligands, water and ions stay.
3. Place the new bilayer. The reference is repeated periodically and cut to the host box. Its midplane goes on the host midplane, and it keeps its own equilibrated thickness. Any box size works, including hexagonal boxes.
4. Remove overlapping lipids, whole lipids only.
   - Any new lipid with a heavy atom within 2 Å of a protein or ligand heavy atom is removed. Raise protein_clash_distance to 3 Å to also rule out lipid tails threaded through aromatic rings.
   - Where copies of the reference meet at the box edge, the fewest are removed so that none overlap.
   - The bilayer is tried at 16 lateral positions. The one that keeps the most lipids is used.
5. Hydrate the new bilayer. The reference water hydrates the new headgroups, up to 10 Å beyond each phosphate plane, and replaces the host water there. Host water and ions close to the protein are kept, including internal waters; the reference water yields to them. Outside that layer, host water and ions are removed only where they come within 2 Å of the new lipids or the reference water.
6. Fix charge and salt. Counterions far from the protein and lipids are removed, or added in place of bulk waters, to reach the target charge, for example after anionic lipids were replaced or added. The reference water brings no ions. Bulk waters away from the protein therefore become salt pairs, using the host's own ion types, until the host's salt concentration is reached again.
7. Write and check. The topology is checked against the coordinates atom by atom, including the net charge. A failed run leaves only its report, with the reason.

Fill the host box completely: membrane-patch

The reference is tiled onto the host box. When the two boxes differ in size, lipids that cross the host box edge meet the periodic images of lipids from the other side, and those are removed: a 12.85 nm patch cut to a 10.6 nm box loses about 20 lipids per leaflet, so the new membrane holds fewer lipids than fit. The run warns when this happens. To fill the box completely, without changing its size:

```
gmxtransplant membrane-patch -i membrane.yaml -o patch/     # same YAML as the membrane run
gmxtransplant slurm-script patch/ --profile dardel --account <project>
sbatch patch/submit.sh                                      # or run patch/sample_run.sh
# then, in membrane.yaml:  reference_gro: /full/path/patch/patch3_npt.gro
gmxtransplant --mode membrane -i membrane.yaml --output results/
```

membrane-patch cuts the reference (bundled POPC or your reference_gro) to the host's lateral box at the reference's own density, with equal leaflets and the reference composition, hydrated by the reference water. It writes patch_start.gro, topol.top and toppar/ (from the host), index.ndx, four mdps and sample_run.sh: minimization, 250 ps NVT and 1 ns NPT with the headgroups held in z, then --ns (default 50) ns of unrestrained NPT. The lateral area stays fixed at the host box (compressibility = 0.0 4.5e-5), so the last frame, patch3_npt.gro, is periodic in exactly the host box: used as reference_gro, only the lipids that overlap the protein are removed.

How many lipids fit: the host's packing does not carry over. Cholesterol (about 0.27 nm²) and POPE (about 0.58 nm²) take less area than POPC (about 0.65 nm²), so a POPC/POPE/cholesterol membrane holds more molecules than the same area of pure POPC.

Output

In <output>/membrane_output/:

File	Contents
step5_input.gro, step5_input.pdb	The new system
topol.top, toppar/	Its topology
index.ndx	CHARMM-GUI groups (SOLU, MEMB, SOLV, SOLU_MEMB) plus Protein, Membrane, POPC_upper, POPC_lower, ...

membrane_report.txt, .json	Host midplane and leaflets; lipids removed per leaflet; the reference used; POPC placed per leaflet; water and ions removed and why; charge and salt; final molecules
equilibration/	The GROMACS equilibration (next section)
view.pml, view.vmd, view.pse	PyMOL/VMD scenes, with the new POPC in its own colour

Equilibration

The removed lipids leave gaps around the protein, and the edges of the hydration layer meet the host water. Run equilibration/ before production. It is a self-contained GROMACS folder (topol.top, toppar/, step5_input.gro, index.ndx, one mdp per stage and sample_run.sh). It follows the CHARMM-GUI Membrane Builder step6 schedule, about 1.9 ns in total:

Stage	Type	Length	Restraints: backbone / side chains / lipid (kJ/mol/nm²)
step6.0_minimization	steepest descent	5000 steps	4000 / 2000 / 1000
step6.1_equilibration	NVT	125 ps (1 fs)	4000 / 2000 / 1000
step6.2_equilibration	NVT	125 ps (1 fs)	2000 / 1000 / 400
step6.3_equilibration	NPT	125 ps (1 fs)	1000 / 500 / 400
step6.4_equilibration	NPT	500 ps (2 fs)	500 / 200 / 200
step6.5_equilibration	NPT	500 ps (2 fs)	200 / 50 / 40
step6.6_equilibration	NPT	500 ps (2 fs)	50 / 0 / 0

The protein is released step by step, so it keeps its OPM placement while the POPC packs around it. The restraints use the POSRES_FC_BB and POSRES_FC_SC blocks in the protein and ligand ITPs, and the POSRES_FC_LIPID (P in z) and DIHRES_FC blocks in the POPC ITP. NPT stages use semi-isotropic C-rescale coupling at 1 bar and 310 K. The report warns if a protein or ligand ITP leaves heavy atoms without a restraint.

Run it on your workstation:

```
cd results/membrane_output/equilibration
./sample_run.sh
```

On a cluster, set the GROMACS commands. For example, in a Slurm job with an MPI build:

```
GMX="srun -n 1 gmx_mpi" MDRUN="srun gmx_mpi mdrun" ./sample_run.sh
```

Then check the stages with python3 analyze_equilibration.py (box, area per phospholipid, P–P thickness, protein C-alpha RMSD and z drift per stage), and continue production from step6.6_equilibration.gro with the same topol.top. gmxtransplant slurm-script results/membrane_output/equilibration --profile dardel --account <project> writes a Slurm job for the whole schedule.

On Dardel (GROMACS 2026.1, one node, 53 ns/day) the D1R example (built with the earlier 3 Å removal distances) ran the full schedule with no LINCS warnings: the box shrank laterally from 12.95 to 12.56 nm as the gaps closed, the P–P thickness settled at 3.83 nm, and the receptor C-alpha RMSD was 0.09 nm after the restraints were released, its centre moving less than 0.1 nm along z. To change a stage, e.g. to lengthen the last one, edit equilibration.stages in the YAML.

All settings

Key	Default	Meaning
host	(required)	System folder
host_gro	step5 GRO	Other coordinates from the host folder
reference_gro	bundled POPC bilayer	Any other equilibrated bilayer GRO
lipid_itps, lipid_forcefield	none	ITPs (and parameters) of reference lipids the host lacks
resname_map	automatic	Reference residue -> moleculetype, when name or atom names do not decide
membrane_center_z	host midplane	Midplane z (Å, host GRO frame)
max_tilt	5.0	Largest leaflet tilt from the xy plane (degrees)
protein_clash_distance	2.0	New lipid heavy atom to protein/ligand heavy atom (Å)
lateral_search	16	Lateral positions of the bilayer tried
water_layer	10.0	Reference hydration beyond each P plane (Å); 0 keeps only host water
protein_hydration_distance	4.0	Host water/ions this close to the protein are kept (Å)
reference_contact_distance	2.0	Allowed contact between reference copies at the box edge (Å)
water_clash_distance	2.0	Host water/ion to new lipid or reference water (Å)

neutralize, target_net_charge	true, 0	Remove or add counterions to reach this charge
ion_exclusion_distance	10.0	Ions this close to the protein are never removed (Å)
lipid_ion_exclusion_distance	5.0	Ions this close to lipids are never removed (Å)
restore_salt	true	Restore the host salt concentration
environment_overrides	{}	Classify unknown molecule types, e.g. MYLIP: lipid
equilibration.enabled	true	Write equilibration/
equilibration.temperature	310.0	K
equilibration.stages	table above	Each stage: name, integrator (steep/nvt/npt), nsteps, dt, backbone, sidechain, lipid

Limits

- The new membrane is entirely the reference's composition: host lipids and sterols are not kept. anchor_count and anchor_separation of earlier versions are ignored.
- The membrane normal must lie along z, the box's third vector.
- The new bilayer keeps the reference's thickness. If it differs from the host's by more than 3 Å, the report warns; NPT equilibration adjusts the water layers.

System-edit modes

These seven modes each change one prepared system (the host) and write a run-ready folder: step5_input.gro/.pdb, topol.top, toppar/, index.ndx with the CHARMM-GUI groups, a report, PyMOL/VMD scenes and, when asked, a GROMACS equilibration folder. The host is a CHARMM-GUI-style folder (topol.top, toppar/, step5_input.gro; host_gro picks another GRO, such as an MD frame). Nothing outside the output folder is changed.

In the desktop application protonate is a main mode; the others are under More modes on the Configuration page, and their examples under More examples on the Examples page. Every mode has a D1 receptor example in examples/<mode>/, built on the apo D1R host of the addbinder example.

Mode	What it does	Example
salt	Sets the salt type and concentration (NaCl, KCl, CaCl2, MgCl2 …)	150 mM KCl instead of NaCl
protonate	Changes protonation states (HSD/HSE/HSP, ASP/ASPP, GLU/GLUP, LYS/LSN, CYS/CYM)	D2.50 charged, H164 -> HSP
mutate	Point mutations with a CHARMM36 topology and a clash-free rotamer	D3.32A, S5.42A
hmr	Hydrogen mass repartitioning for 4 fs time steps	3.024 u hydrogens
resize	Makes the water layer thicker with copies of the host's bulk water	+20 Å
flood	Adds many copies of a small molecule to the water (ligand flooding, cosolvent probes)	10 dopamine
oligomer	Adds copies of the membrane protein to the same membrane	D1R dimer lattice

```
gmxtransplant --show-example salt > salt.yaml      # set host (and the mode's settings)
gmxtransplant --mode salt -i salt.yaml --dry-run
gmxtransplant --mode salt -i salt.yaml --output results/
```

Every input path must be given in full (the bundled examples are the exception). The output goes to <output>/<mode>_output/ unless output_dir is set.

Settings every mode shares

Key	Meaning
host	The prepared system folder (required).
host_gro	Optional coordinates instead of the folder's step5_input.gro.
environment_overrides	Molecule types the automatic classifier does not know, e.g. {MYLIP: lipid}.
neutralize, target_net_charge	After the change, counterions far from the protein are removed or added (replacing bulk waters) to reach the target charge (default 0). The host's own monovalent ions are used.
equilibration	enabled: true writes equilibration/ (below). Off by default; the examples turn it on.
output_dir	Output folder name or path.

To replace the membrane (pure POPC, or any equilibrated bilayer from a GRO file), use the membrane mode (MEMBRANE.md).

salt

concentration (mol/L of formula units), cation and anion (moleculetype names: CHARMM36 SOD POT LIT CES RUB CAL MG BAR ZN2 CD2, CLA). With replace_existing: true the host's bulk ions are removed first; keep_bound_distance (Å) keeps ions bound to the protein, e.g. a structural Na+. New ions replace random bulk waters at least solute_distance from the protein and ligands, membrane_distance from the lipids and ion_spacing apart. The concentration is units / waters × 55.51, counting the waters left after replacement; CaCl2 counts once per Ca^{2+}. An ion type the host lacks comes from ion_itps or the bundled CHARMM36 ions, with its atom type and NBFIX pairs added to forcefield.itp; nothing the host defines is changed.

protonate

```
protonate:
  host: /path/to/system
```

```
  residues:
    - {molecule: PROA, resid: 164, state: HSP}
    - {molecule: PROA, resid: 70, state: ASP}
```

resid is the residue number in the protein ITP. A residue's present state is read from its atoms, not its name (CHARMM-GUI names a protonated aspartate ASP, with HD2). States: HSD HSE HSP ASP ASPP GLU GLUP LYS LSN CYS CYM; a change must stay within one residue (use mutate otherwise).

The residue's atoms get the CHARMM36 types and charges of the new state; every bonded term that touches the residue beyond its backbone N and C is regenerated the way pdb2gmx does for CHARMM36: bonds, impropers and CMAP from the residue template, angles, proper dihedrals and 1-4 pairs from the bond graph. This reproduces CHARMM-GUI's own ITP exactly for every standard residue (tested). New hydrogens are placed from the CHARMM internal coordinates (within 0.1 Å of CHARMM-built positions); position restraints follow CHARMM-GUI (POSRES_FC_BB on N, CA, C, O; POSRES_FC_SC on other heavy atoms). Parameters the new state needs and the host forcefield.itp lacks (e.g. the HSP atom types) are added from the bundled CHARMM36 library. Every copy of an edited moleculetype is changed. Terminal, capped and disulfide-bonded residues are refused. Water within water_clash_distance (2 Å) of a new atom is removed.

mutate

```
mutate:
  host: /path/to/system
  mutations:
    - {molecule: PROA, resid: 103, to: ALA}
```

to is a CHARMM36 residue (ALA ARG ASN ASP ASPP CYS GLN GLU GLUP GLY HSD HSE HSP ILE LEU LYS LSN MET PHE PRO SER THR TRP TYR VAL). The backbone (N, HN, CA, HA, C, O) keeps its coordinates, and CB when both residues have one. The side chain is built from the CHARMM internal coordinates; its rotamer is chosen from staggered chi angles (-60°, 180°, 60°; for planar groups ±90°, 0°, 180°) by the smallest steric overlap with the rest of the system, water and ions excluded. The report gives the chi angles and the score. The topology is rebuilt as for protonate. Water and ions within water_clash_distance (2.5 Å) of a new atom are removed; lipids within lipid_clash_distance (2 Å) are removed whole (remove_clashing_lipids: false only warns). Minimize before MD. PDBFixer side chains are planned (roadmap (PDBFIXER_ROADMAP.md)).

hmr

Every hydrogen bonded to a heavy atom gets hydrogen_mass (3.024 u), taken from that heavy atom, so each molecule keeps its mass; rigid water is left unchanged (skip_water). With copy_run_inputs the host's mdp files are copied and the production stage (step7) gets time_step (0.004 ps) with the same simulated length. Use 4 fs only with constraints = h-bonds.

resize

add_water (Å) is added to the box along z. The water above and below a membrane is one layer joined through the periodic boundary, so the new water goes in its middle: the molecules move so the bilayer is centred in the taller box, and the gap is filled with copies of the host's own bulk water slab (water and ions, at the host's density and salt), taken at least slab_margin (4 Å) from the protein and lipids. Copies within water_clash_distance (2.4 Å) of another molecule are dropped at the seams, so the new layer starts about 10% below bulk density; the NPT equilibration closes the gap. Copied ions are dropped as needed to keep the charge.

flood

molecule_coordinates, molecule_itp and molecule_forcefield (the forcefield.itp generated with a CGenFF ligand) describe the molecule; copies (or concentration in mol/L of the water) how many. Each copy gets a random position and orientation in the water (side: both | upper | lower), at least min_solute_distance (10 Å) from the protein and ligands, min_membrane_distance (5 Å) from the lipids and min_copy_distance (8 Å) from other copies, periodic images included. Water and ions within water_clash_distance of a copy are removed. The index group Flood_<name> holds every copy.

oligomer

```
oligomer:
  host: /path/to/system
  copies:
    - {shift: [64.76, 64.76], rotation: 180.0, flip: false}
```

Each copy of the host's proteins (and their ligands, include_ligands) is turned by rotation about the membrane normal through the protein centre, optionally flipped upside down about an in-plane axis through the membrane midplane, and shifted by shift (x, y in Å). Every copy must stay min_protein_gap (10 Å) from every other copy and all periodic images, or the run stops with the distance found. Lipids within lipid_clash_distance (3 Å) of a copy are removed whole, water and ions within water_clash_distance. The index groups Original and Copies separate them. In a small box, try a diagonal shift (half the box in x and y) before a shift along one axis.

The equilibration folder

equilibration/ holds the system, topol.top, toppar/, index.ndx, one mdp per stage and sample_run.sh. The default stages are the CHARMM-GUI Membrane Builder step6 schedule (minimization, two NVT and four NPT stages, 1.875 ns), releasing the backbone, side-chain and lipid restraints step by step; stages can be listed in equilibration.stages. Run it with

```
cd results/salt_output/equilibration
GMX="gmx" ./sample_run.sh
python3 analyze_equilibration.py
```

analyze_equilibration.py (numpy only) reads every finished stage and prints the box, the projected area per phospholipid of each leaflet, the P–P thickness, the protein C-alpha RMSD from the input (best fit, periodic images taken into account) and the drift of the protein centre along z.

On a cluster, gmxtransplant slurm-script writes submit.sh next to sample_run.sh:

```
gmxtransplant slurm-script results/salt_output/equilibration --profile dardel --account naiss2025-3-21
sbatch results/salt_output/equilibration/submit.sh
```

Profiles: generic, dardel (PDC modules, srun gmx_mpi, 128 ranks) and gpu; --partition, --module, --gmx, --mdrun, --nodes, --ntasks-per-node, --cpus-per-task, --time and --gpus override them. Nothing is submitted.

The GROMACS check

After every successful run, every mode checks what it wrote with gmx grompp -maxwarn 0 (a plain minimization mdp, no restraints) when GROMACS is found: GMXTRANSPLANT_GMX, GMX, then gmx, gmx_mpi on PATH. --gromacs-check always requires it, never skips it. A rejected system keeps its files and the run exits with code 8. Any folder can be checked later:

```
gmxtransplant check-gromacs results/salt_output results/mutate_output
```

Validation

All seven examples (and both membrane examples) were built on Dardel (GROMACS 2026.1) with --gromacs-check always: grompp passed each with 0 warnings and 0 notes; the step6.0 minimization and 5 ps of the first restrained NVT stage ran without LINCS warnings, and the HMR system ran 10 ps at 4 fs without LINCS warnings.

Comparison views

Each successful assembly exports view.pml, view.vmd, and visualization/ alongside its coordinates. A real view.pse session is generated automatically when PyMOL is installed; PyMOL is found on the PATH, in the running Python environment, or on macOS in /Applications, and GMXTRANSPLANT_PYMOL can point to any other copy. PyMOL and VMD are optional external viewers; neither becomes a CLI or GUI dependency. Minimization preparation shows the prepared input, not an unperformed minimization result.

Use Open in PyMOL or Open in VMD in GUI results, or run from the output folder:

```
pymol view.pml
vmd -e view.vmd
```

Every mode that produces a visualization writes view.vmd and view.pse into its output folder, alongside view.pml and the visualization/ assets, so the assembled system can be reopened later without rerunning the pipeline. view.pse is written during the run when PyMOL is on PATH; when it is not, the run says so and opening view.pml saves view.pse. The PSE is self-contained. When moving a scripted scene, copy the entire output folder, including visualization/, and launch from that folder. VMD locates its assets relative to the .vmd file. For Python-script loading in PyMOL, use run visualization/load.py, module.

Component	Color	Initially visible
Incoming protein	Blue	Yes
Retained protein	Slate	Yes
Incoming ligand	Magenta	Yes
Retained ligand (topology-classified transplant mode)	Purple	Yes
Incoming cholesterol	Gold	Yes
Retained cholesterol	Teal	Yes
Retained membrane	Pale gray	Yes
Original replaced protein	Orange	No
Original replaced ligand	Orange	Yes
Removed environment	Red	No
Reference input	Amber	No
Aligned transplant input	Cyan	No
Water / ions	Pale blue	No

Toggle named objects in PyMOL or molecules in VMD. The old ligand is a comparison overlay, not part of the final molecular system. Full input overlays and removed environment are initially hidden to avoid obscuring the result. visualization/LEGEND.txt and scene.json list the roles and atom counts actually present in that run.

Source overlays use the same rigid transformation as assembly, in the final protein's coordinate frame. Ligand mode shows the positioned incoming ligand; cholesterol mode shows the converted, aligned experimental structure. Box repair, when enabled, is already reflected in the reference overlay. These views do not apply a second fit, wrap molecules, or change scientific coordinates.

Inserted membership comes from the actual inserted atom block, rather than guessing by residue name: an old and a new ligand with the same residue name remain distinct. Two-folder mode additionally uses exact topology molecule categories. Other modes use their standard residue classification for display; unknown retained components are labeled “Other retained molecules.” Viewer files are for comparison, not simulation inputs; use the validated final GRO and topology for simulation.

Viewer export failures are reported without invalidating successfully written scientific outputs. Details remain in View Command Progress. The generated scripts work without network access on supported viewer installations on Linux, WSL, and macOS.

Portable and offline use

Install from the source directory:

```
python3 -m pip install .
```

Create and edit a configuration in your job directory:

```
gmxtransplant --show-example protein > protein_replace.yaml
gmxtransplant --mode protein -i protein_replace.yaml --dry-run
gmxtransplant --mode protein -i protein_replace.yaml --prepare-minimization
```

Use modes lig or chl for ligand replacement or cholesterol restoration. Omit the preparation flag and leave minimization.enabled: false for assembly alone.

To prepare an existing matching system:

```
gmxtransplant prepare-minimization --coordinates step5_input.gro --topology topol.top
```

Both workflows create one openmm_minimization/ folder. Preparation copies and audits the inputs; it does not minimize or submit a job. The folder includes all topology dependencies, the standalone runner, settings, and generic CPU/GPU SLURM templates.

Move the whole folder to your execution machine, activate your Python environment, and review the settings:

```
cd openmm_minimization
python3 -m pip install -r requirements.txt
bash run.sh
```

The runner relaxes clashes with harmonic restraints on protein and ligand heavy atoms and lipid head groups (polar P, O, N and S atoms). Lipid tails, water, ions, and hydrogens relax freely. Classify custom lipids with minimization.restraint_residue_classes so they are not treated as ligands.

For SLURM, edit and submit either submit_cpu.slurm or submit_gpu.slurm from this folder. Inspect results/minimized.gro and the convergence reports before continuing. Existing inputs and results are never overwritten.

See README.md (../../README.md) for all modes, settings, and limitations.